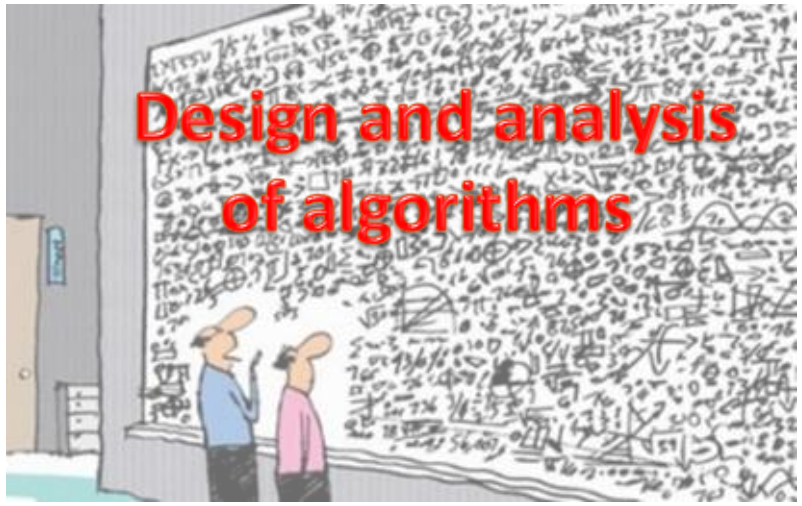


Greedy algs

- Minimum Spanning Trees

Objective

- Greedy algorithms for spanning trees.
  - Animations: https://visualgo.net/en/mst?slide=1
- The Union-Find algorithm.
- Optional: the $m \log^* n$ proof. https://www.cs.cmu.edu/~15451-f16/lectures/lec05-unionfind.pdf.
- DPV Chapter 5 (we don't cover the material in section 5.3).

What is “greedy”?

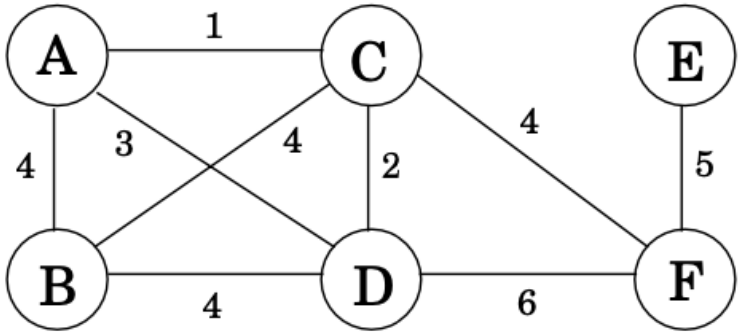
- Myopic behavior is easy and convenient, attractive for designing algorithms.
- **Greedy** algorithms: build up a solution piece by piece, always choosing the next piece that offers the most obvious and immediate benefit.
- This is not always sensible, but there are some computational tasks for which it is optimal to be greedy.

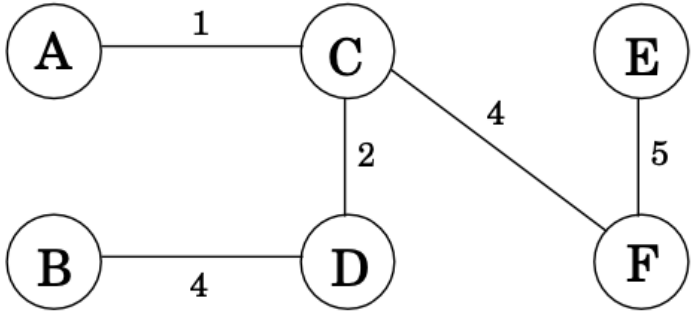
Minimum spanning trees

In this chapter, *tree* is an **undirected** graph that is connected and acyclic

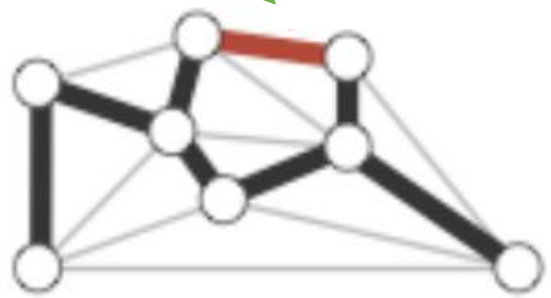
- **(Spanning) tree** of an **undirected** **connected** graph $G = (V, E)$: a subgraph of G that is connected and acyclic and covers all nodes of G.
  - Many different spanning trees can be constructed from a given graph G.
- Minimum spanning tree: a spanning tree with minimum total weight.

is it equivalent with:
"minimum weight connected subgraph that covers all nodes" ?

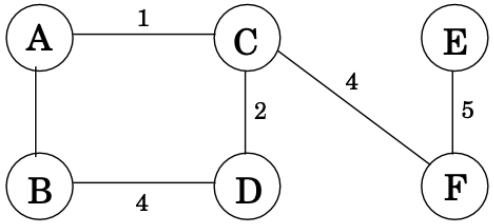


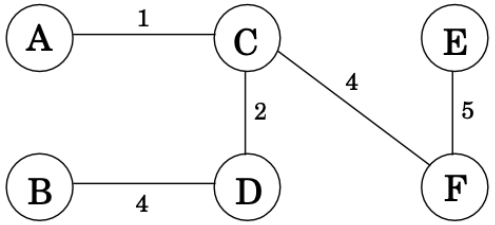


Property: Given a spanning tree (black edges), add any new edge (red): creates a cycle.
Take out any black edge from the cycle, keep the red: remains a tree.



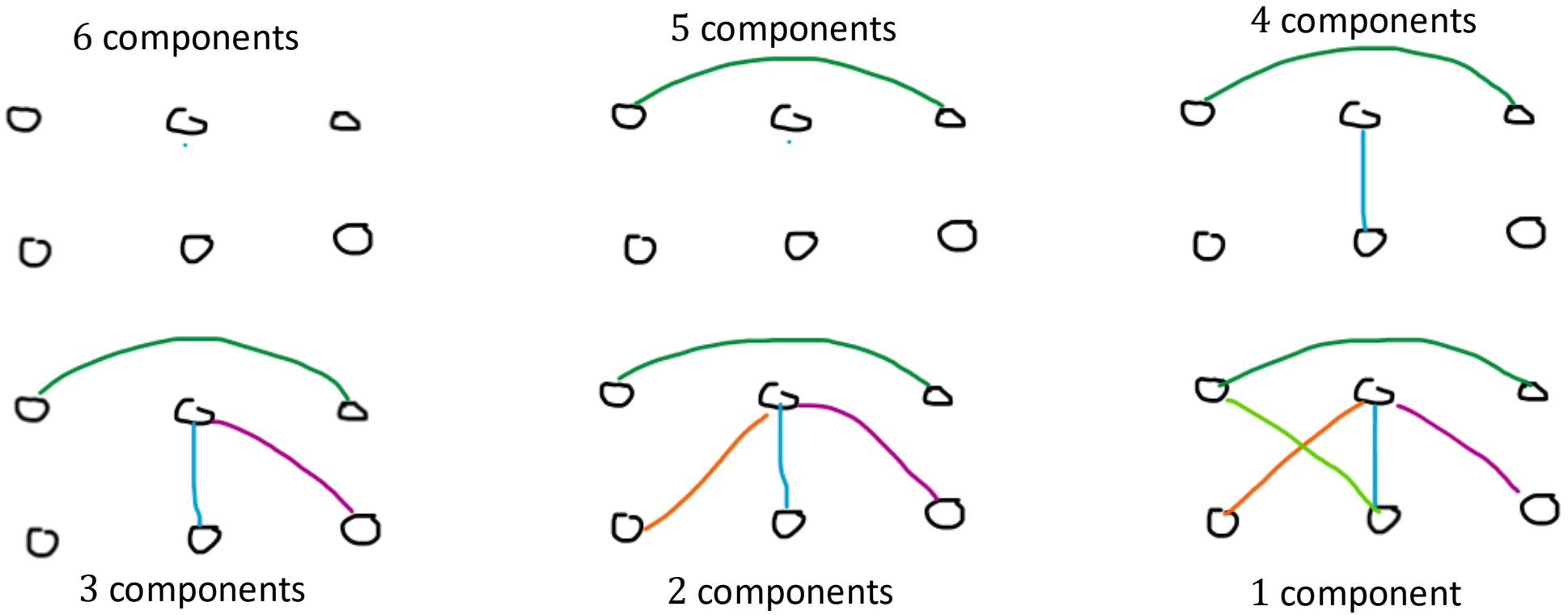
Properties





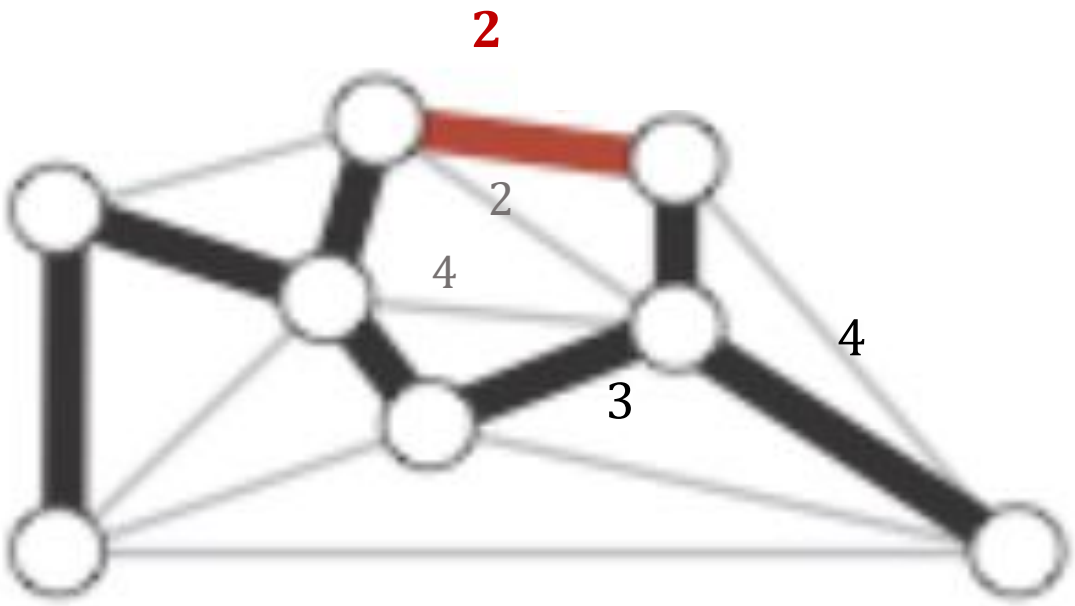
- Property 1: Removing a cycle edge cannot disconnect a graph that was connected.
- Property 2: A tree on n nodes has $n - 1$ edges.
  - **Algorithm**: start with all nodes disconnected, defining each a connected component. Keep adding each edge (u, v) making sure u and v are in separate connected components, hence reducing the total number of connected components by one.
- Property 3: Any connected undirected graph $G = (V, E)$ with $|E| = |V| - 1$ is a tree.
- Property 4: An undirected graph is a tree if and only if there is a unique path between any pair of nodes.

We start with a connected graph. Consider any sequence of adding edges to the nodes until the graph is connected without loops (acyclic).
Start with $n = 6$ connected components (each node is a simplest connected component). Adding a new edge that does not create a loop reduces the # of connected components by 1 (since it connects two components to make a larger one). We stop when we have 1 connected component. Hence, we cannot have more than $n - 1 = 5$ edge additions. The resulting graph is a tree since it is connected and acyclic. It is connected because it was originally connected.

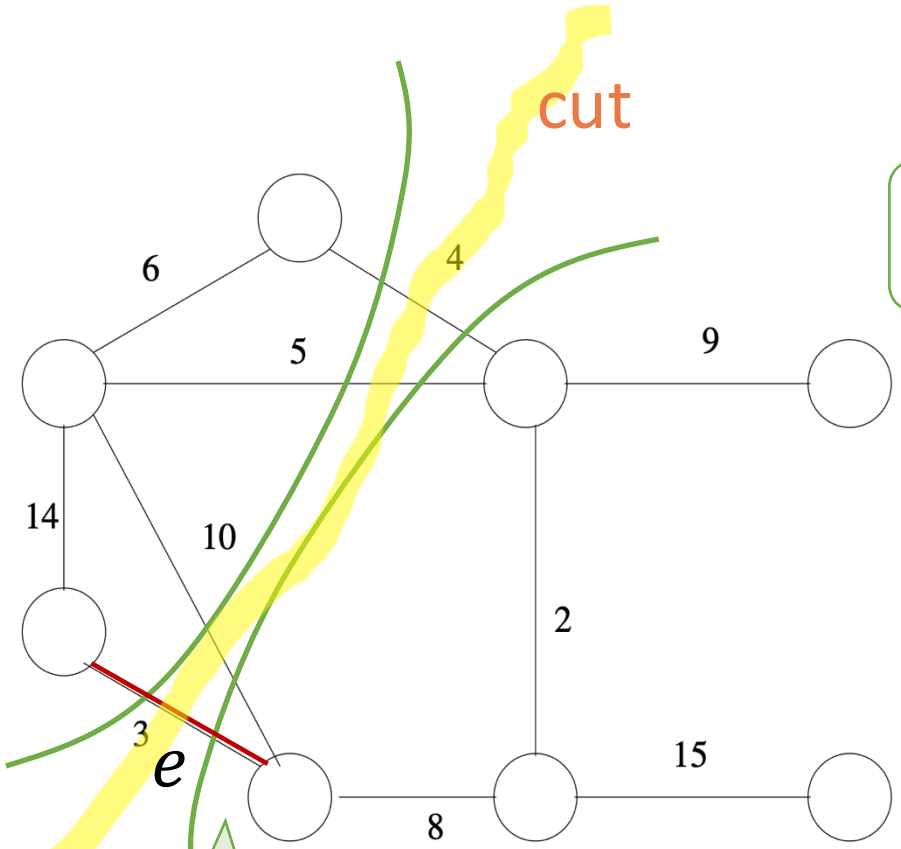


Adding an edge to a ST creates a cycle

- Add the red edge to a spanning tree: creates a cycle.
- Remove any edge from that cycle → get a spanning tree.
- If I remove the heaviest edge: improve total weight of the tree.



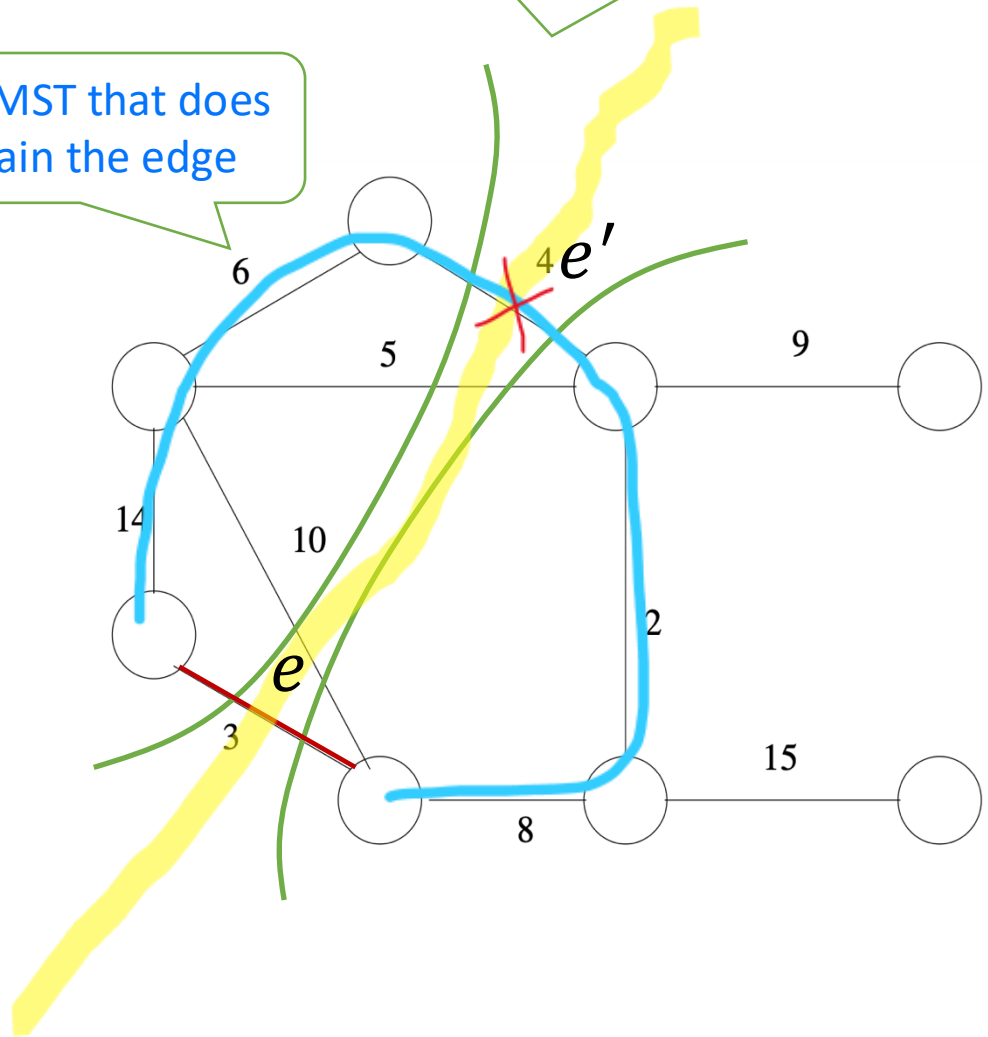
`Cutting' lemma 1



Any lightest edge across the cut must be in some MST T

cycle of a MST that does not contain the edge

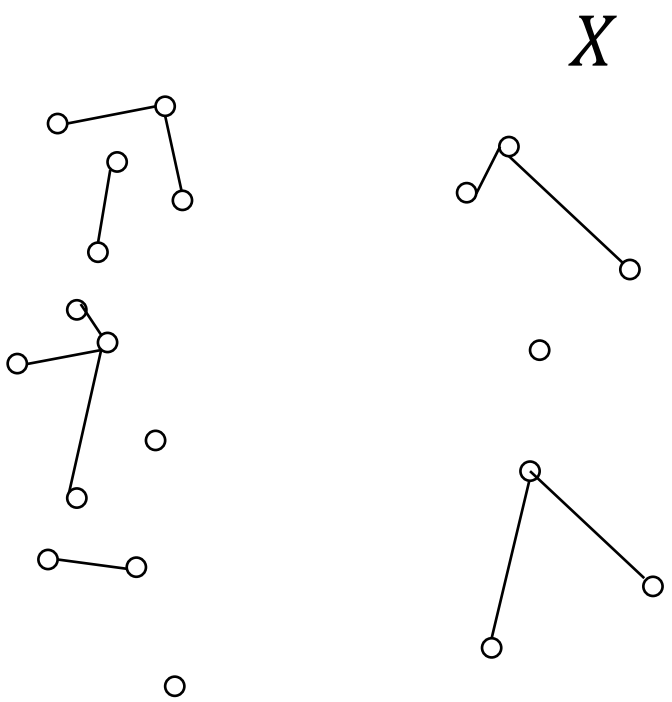
Substitute e for e' in T. This T' must also be an MST



`Cutting' lemma 2

can eventually become an MST by adding more edges

- Given $G = (V, E)$, a set $X \subseteq E$ is "safe" if there is an MST $T' = (V, E')$, s.t., $X \subseteq E'$.
- A cut C "respects" X if $C \cap X = \emptyset$.
- **Lemma**: Suppose the set of edges X are safe. Pick any cut C that *respects* X and let e be a lightest edge across C. Then $X \cup \{e\}$ is safe (i.e., is part of some MST).

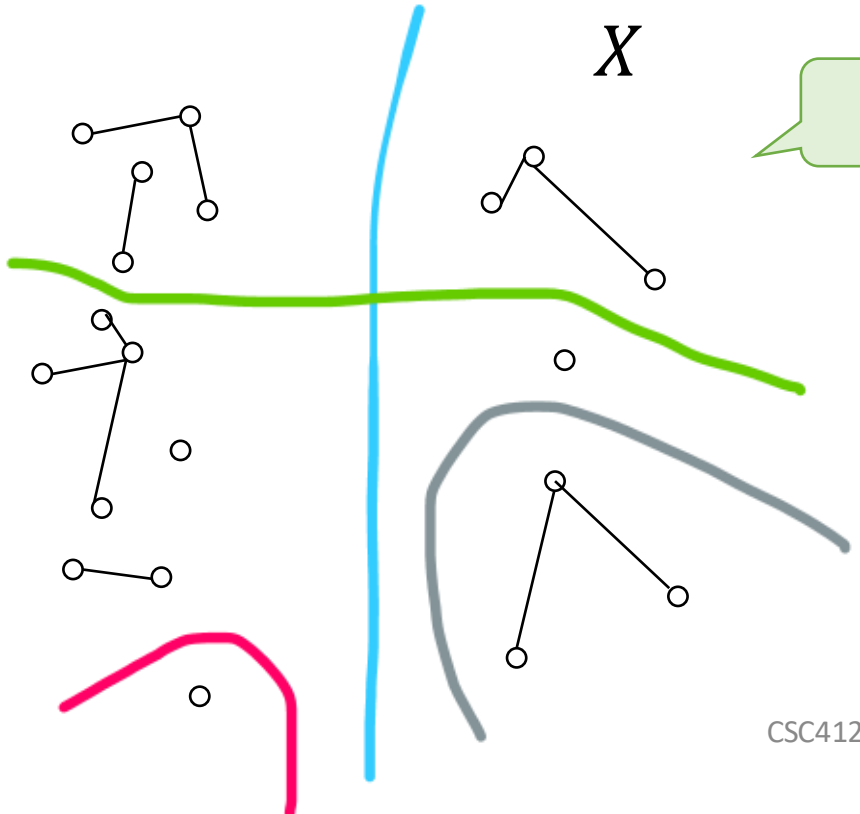




`Cutting' lemma 2

can eventually become an MST by adding more edges

- Given $G = (V, E)$, a set $X \subseteq E$ is "safe" if there is an MST $T' = (V, E')$, s.t., $X \subseteq E'$.
- A cut C "respects" X if $C \cap X = \emptyset$.
- **Lemma**: Suppose the set of edges X are safe. Pick any cut C that *respects* X and let e be a lightest edge across C. Then $X \cup \{e\}$ is safe (i.e., is part of some MST).

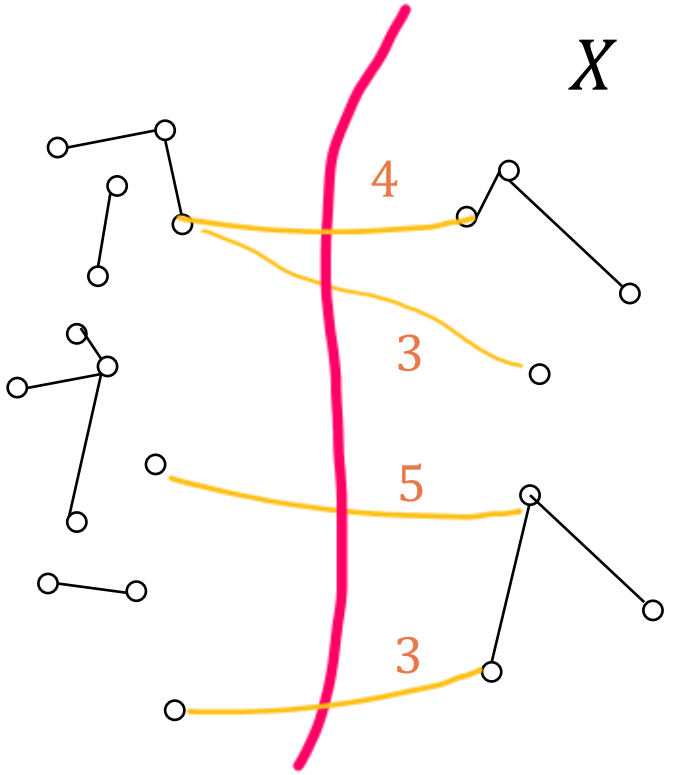


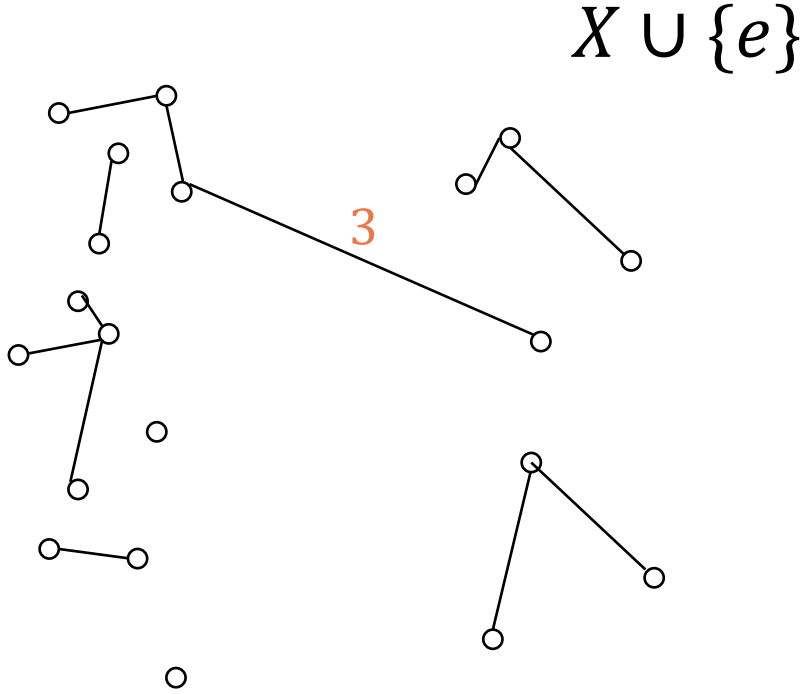
safe cuts

`Cutting' lemma 2

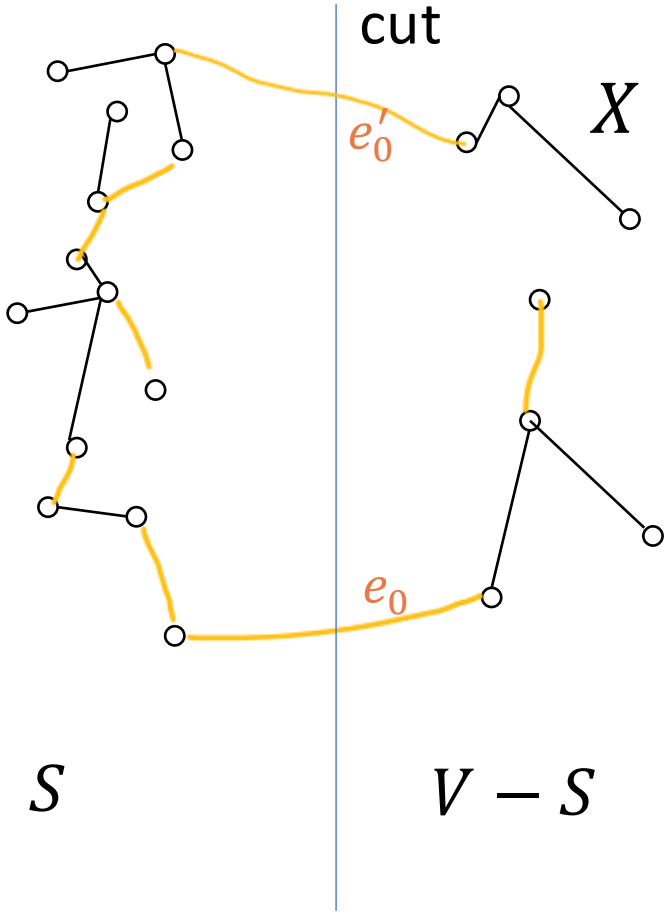
can eventually become an MST by adding more edges

- Given $G = (V, E)$, a set $X \subseteq E$ is "safe" if there is an MST $T' = (V, E')$, s.t., $X \subseteq E'$.
- A cut C "respects" X if $C \cap X = \emptyset$.
- **Lemma**: Suppose the set of edges X are safe. Pick any cut C that *respects* X and let e be a lightest edge across C. Then $X \cup \{e\}$ is safe (i.e., is part of some MST).

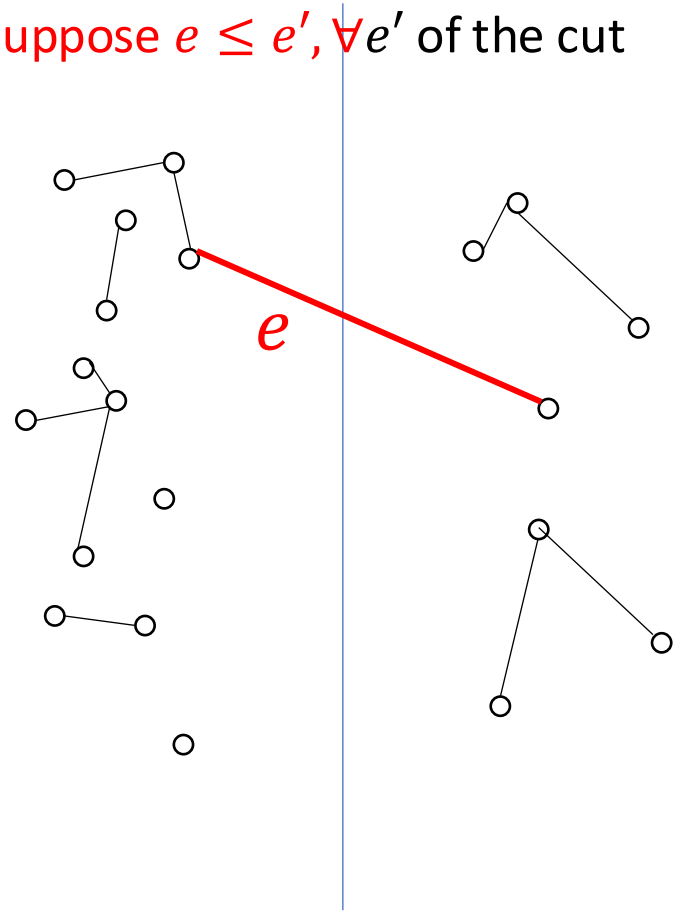


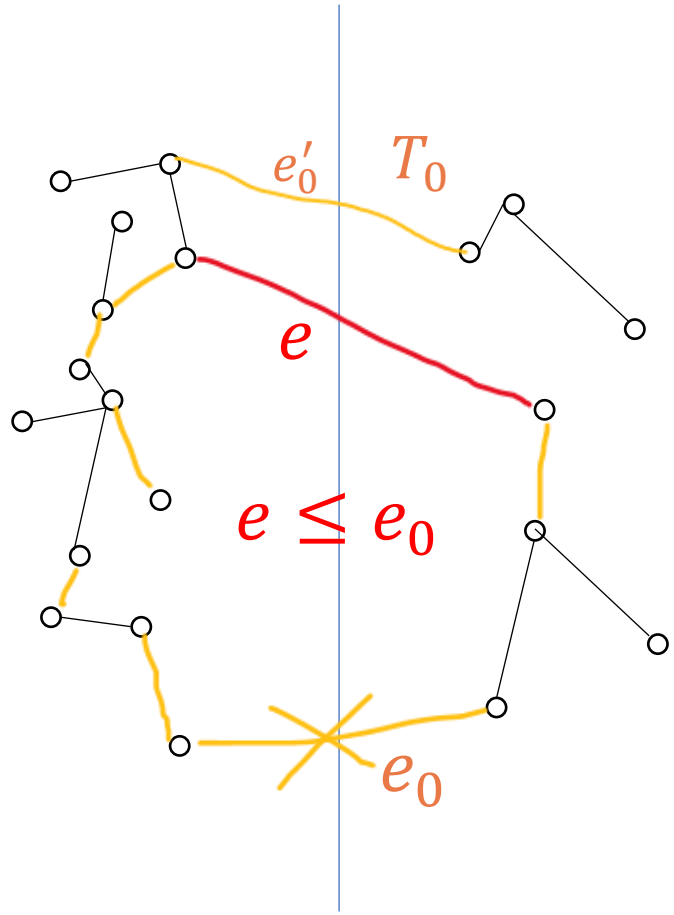


$MST\ T_0 = (V, E_0), X \subseteq E_0$



suppose $e \leq e'$, $\forall e'$ of the cut



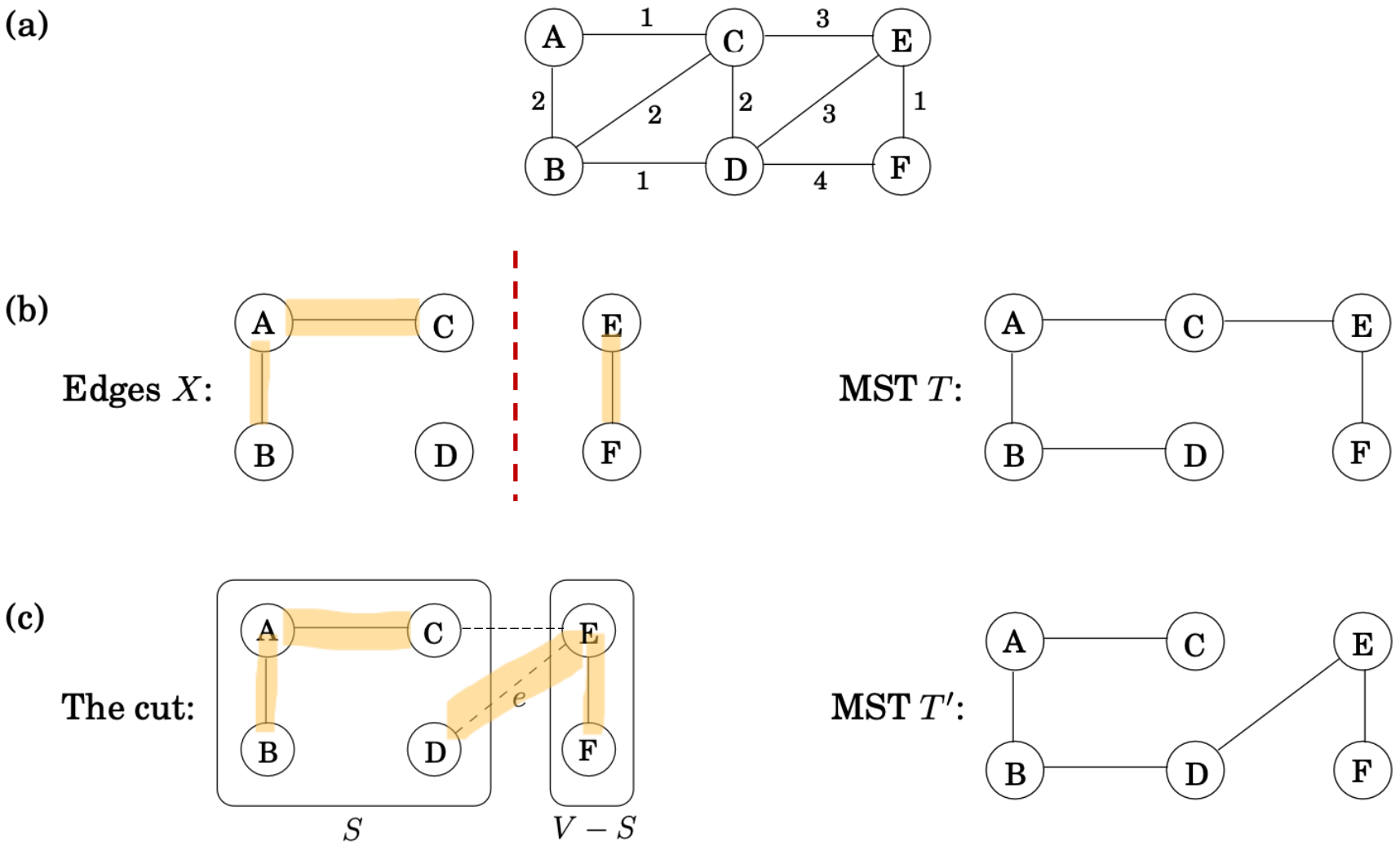


$X \subset E'$ is safe.

e: lightest edge of cut
Prove that $X + e$ is safe.

Add e to the MST T_0, create a cycle that crosses the cut at e_0, take out e_0. The set $E' = E_0 - e_0 + e$ is an MST, hence, $X + e$ is safe.

Example



Kruskal's minimum spanning tree algorithm

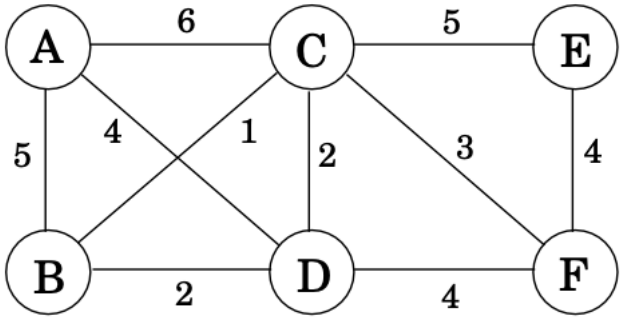
- Start with the empty graph, then select edges from E according to the following "greedy" rule:

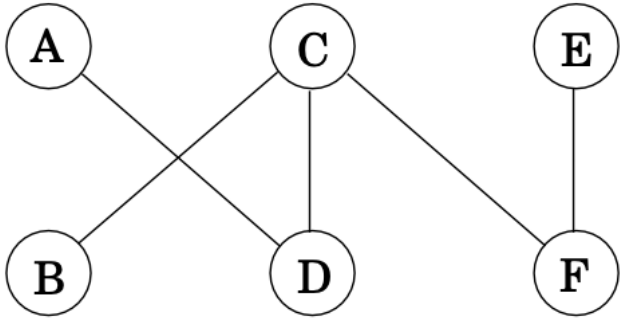
  Repeatedly add the next lightest edge that doesn't produce a cycle.

- Correctness: because of "cut property" (next slide).

order of edges according to weight

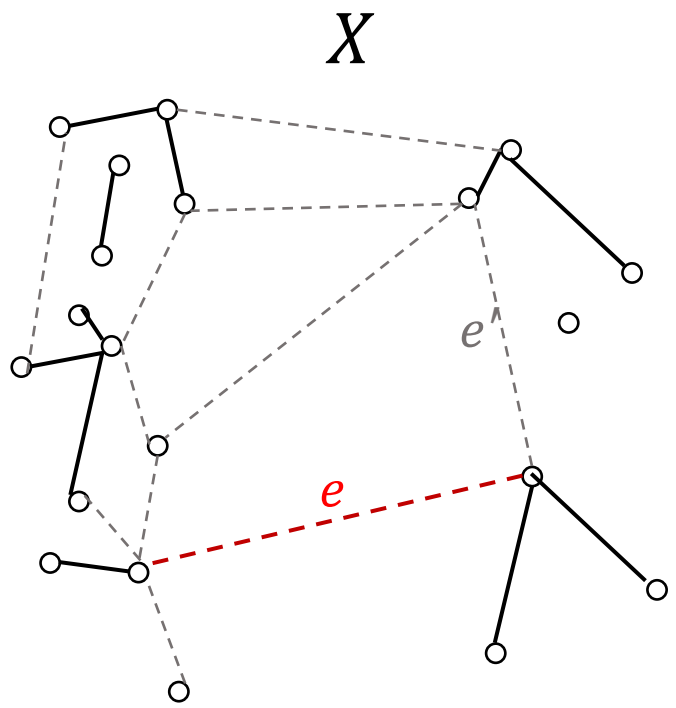
$$B-C,\ C-D,\ B-D,\ C-F,\ D-F,\ E-F,\ A-D,\ A-B,\ C-E,\ A-C.$$





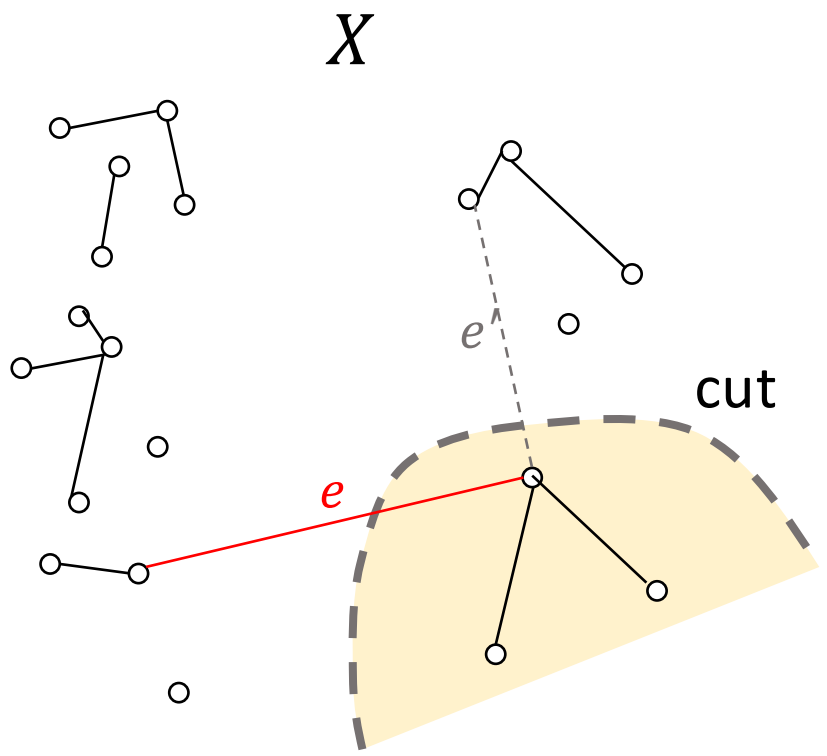
Why Kruskal is correct?

- e: next lightest edge available in $E - X$ that does not form a loop.
- Consider the cut that isolates one of the components that e touches from the rest of the components in X.
- Any other edge e' that crosses the cut is not added so far, hence must be $e' \geq e$.
- By the cutting lemma, $X \cup \{e\}$ is safe.



Why Kruskal is correct?

- e: next lightest edge available in $E - X$ that does not form a loop.
- Consider the cut that isolates one of the components that e touches from the rest of the components in X.
- Any other edge e' that crosses the cut is not added so far, hence must be $e' \geq e$.
- By the cutting lemma, $X \cup \{e\}$ is safe.

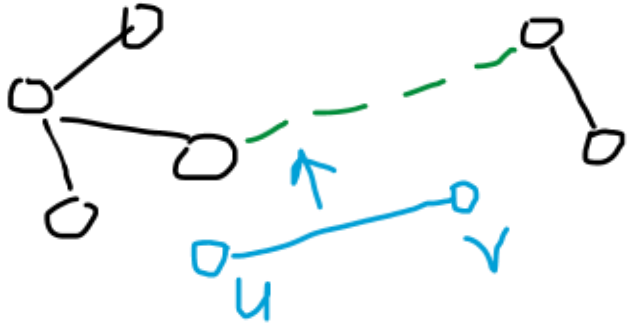


Implementation of Kruskal's algorithm

- At each stage, the algorithm chooses an edge to add to its current partial solution. To do so, it needs to test each candidate edge (u, v) to see whether the endpoints u and v lie in different components.

otherwise, the edge produces a cycle.

- Once an edge is chosen, the corresponding components need to be merged.
- What kind of data structure supports such operations?
- makeset(x), find(x), union(u, v).




```
procedure kruskal(G, w)
for all u ∈ V:
    makeset(u)

X = {}
Sort the edges E by weight
for all edges {u, v} ∈ E, in increasing order of weight:
    if find(u) ≠ find(v):
        add edge {u, v} to X
        union(u, v)
```

Given a connected undirected graph G with weights w

Union-find

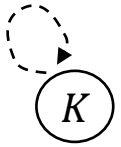
Representing sets $\{K\}, \{E, B\}, \{F, C, D, A, G, H\}$

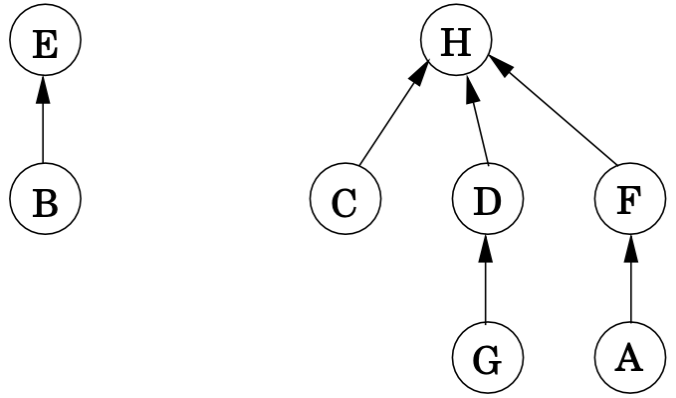
```
procedure makeset(x)
π(x) = x
rank(x) = 0

function find(x)
while x ≠ π(x) :  x = π(x)
return x

procedure union(x, y)
r_x = find(x)
r_y = find(y)
if r_x = r_y:  return
if rank(r_x) > rank(r_y):
    π(r_y) = r_x
else:
    π(r_x) = r_y
    if rank(r_x) = rank(r_y) :  rank(r_y) = rank(r_y) + 1
```

Represent a set as a tree. The root of the tree gives the identity to the set.





Since find(x) takes time proportional to depth of the node, make trees of small depth!

To reduce depth: make shorter tree point to root of taller tree.

In Kruskal we do first two *finds* to find r_x, r_y; if $r_x \neq r_y$ then we do union(r_x, r_y) using the root nodes we already found, so we do each *find* once.

Rank is increased $+1$ only by merging two trees of equal rank

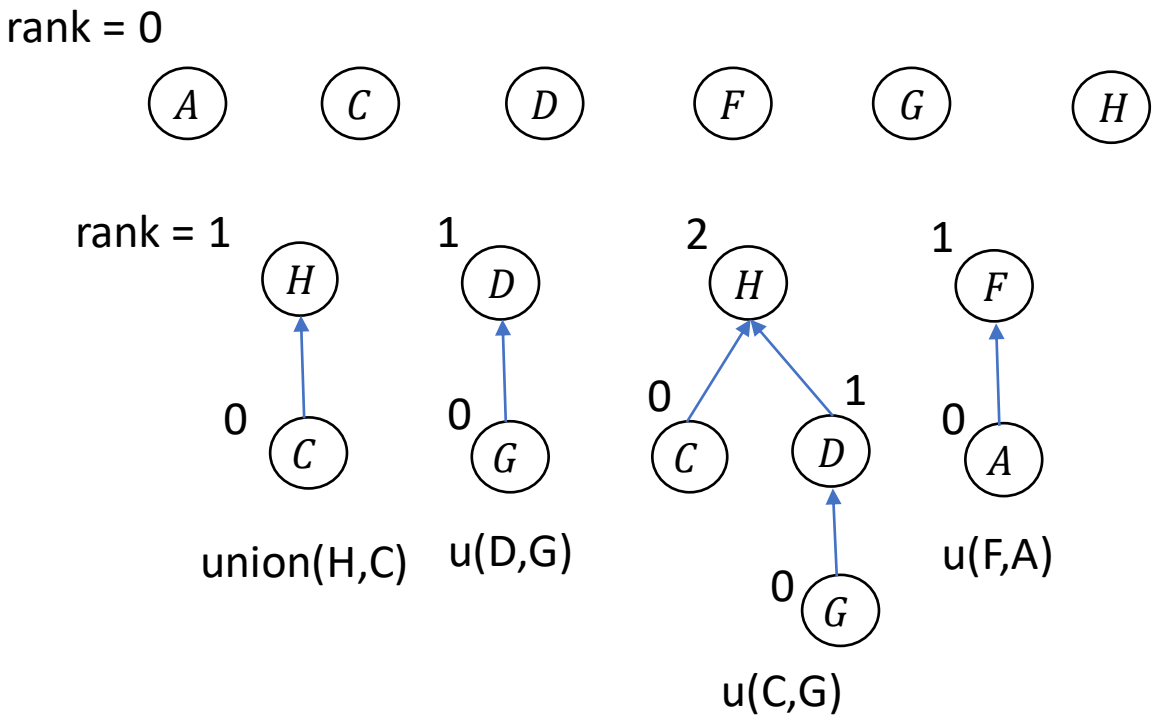
Union-find

```
procedure makeset(x)
π(x) = x
rank(x) = 0

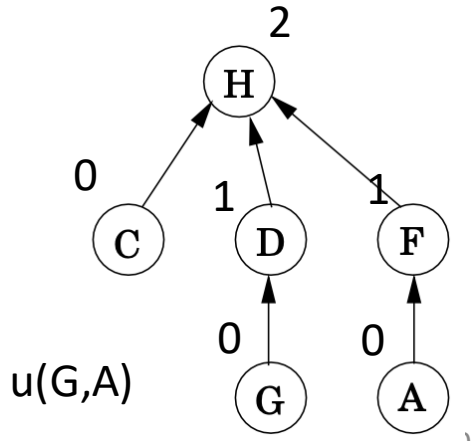
function find(x)
while x ≠ π(x) :  x = π(x)
return x

procedure union(x, y)
r_x = find(x)
r_y = find(y)
if r_x = r_y:  return
if rank(r_x) > rank(r_y):
    π(r_y) = r_x
else:
    π(r_x) = r_y
    if rank(r_x) = rank(r_y) :  rank(r_y) = rank(r_y) + 1
```

Example of building $\{F, C, D, A, G, H\}$



rank k: at least 2^k nodes in subtree. Not related to depth.



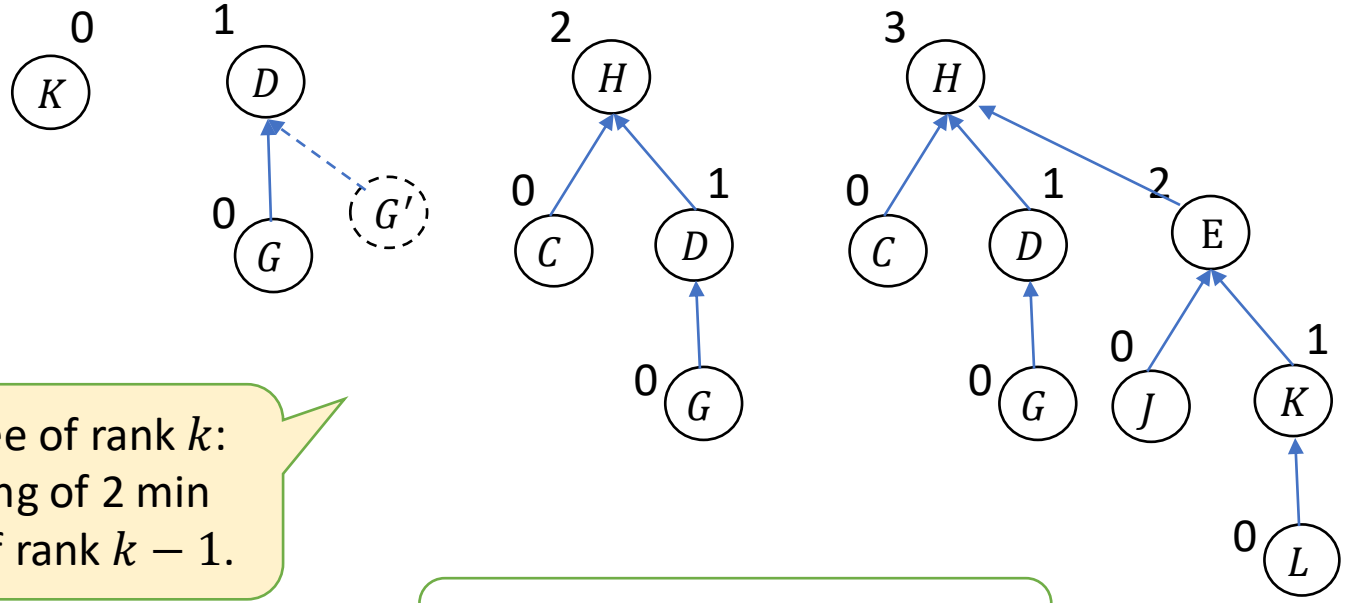
The operations

```
procedure makeset(x)
π(x) = x
rank(x) = 0

function find(x)
while x ≠ π(x) :  x = π(x)
return x

procedure union(x, y)
r_x = find(x)
r_y = find(y)
if r_x = r_y:  return
if rank(r_x) > rank(r_y):
    π(r_y) = r_x
else:
    π(r_x) = r_y
    if rank(r_x) = rank(r_y) :  rank(r_y) = rank(r_y) + 1
```

Minimum # of nodes for sets of rank 0, 1, 2, 3



min tree of rank k: merging of 2 min trees of rank $k-1$.

Any root node of rank k has $\geq 2^k$ nodes in its tree.

A root node with rank k is created by the merger of two trees with roots of rank $k-1$.

The operations

```
procedure makeset(x)
π(x) = x
rank(x) = 0

function find(x)
while x ≠ π(x) :  x = π(x)
return x

procedure union(x, y)
r_x = find(x)
r_y = find(y)
if r_x = r_y:  return
if rank(r_x) > rank(r_y):
    π(r_y) = r_x
else:
    π(r_x) = r_y
    if rank(r_x) = rank(r_y) :  rank(r_y) = rank(r_y) + 1
```

$O(\log n)$

$O(\log n)$

$O(1)$

Property 1: For any $x, rank(x) < rank(\pi(x))$.

Property 2: Any root node of rank k has at least 2^k nodes in its tree.

Property 3: If there are n elements overall, there can be at most $\frac{n}{2^k}$ nodes of rank k.

⇒ maximum possible rank = $\log n$

= maximum depth of any tree.

- Only a single root node has the max rank, hence, solve for k: $\frac{n}{2^k} = 1$.
- Since rank strictly decreases from root to leaves, there cannot be more than max rank = $\log n$ levels.

Implementation

```
procedure kruskal(G, w)
for all u ∈ V:
    makeset(u)

X = {}
Sort the edges E by weight
for all edges {u, v} ∈ E, in increasing order of weight:
    if find(u) ≠ find(v):
        add edge {u, v} to X
        union(u, v)
```

$\Theta(n)$

Since graph is connected:
$n - 1 \leq m \leq n(n-1)$
$\Rightarrow \Theta(\log n) = \Theta(\log m)$

$\Theta(m \log m)$
$= \Theta(m \log n)$

m times

$\Theta(\log n)$

$\Theta(m \log n) + \Theta(m \log n)$

Complexity: $O(|E| \log|V| + |E| \log|V|)$

sorting

find+ union

In Kruskal we do first two finds to find r_x, r_y; if $r_x \neq r_y$ then we do union(r_x, r_y) using the root nodes we already found, so we do each find once.

can we do better if sorting is $\Theta(|E|)$?

Implementation

```
procedure kruskal(G, w)
for all u ∈ V:
    makeset(u)

X = {}
Sort the edges E by weight
for all edges {u, v} ∈ E, in increasing order of weight:
    if find(u) ≠ find(v):
        add edge {u, v} to X
        union(u, v)
```

$\Theta(n)$

Since graph is connected:
$n - 1 \leq m \leq n(n-1)$
$\Rightarrow \Theta(\log n) = \Theta(\log m)$

$\Theta(m \log m)$
$= \Theta(m \log n)$

m times

$\Theta(\log n)$

with path compression
$\log^* |V|$

$\Theta(m \log n) + \Theta(m \log n)$

Complexity: $O(|E| \log |V| + |E| \log |V|)$

sorting

find+ union

can we do better if sorting is $\Theta(|E|)$?

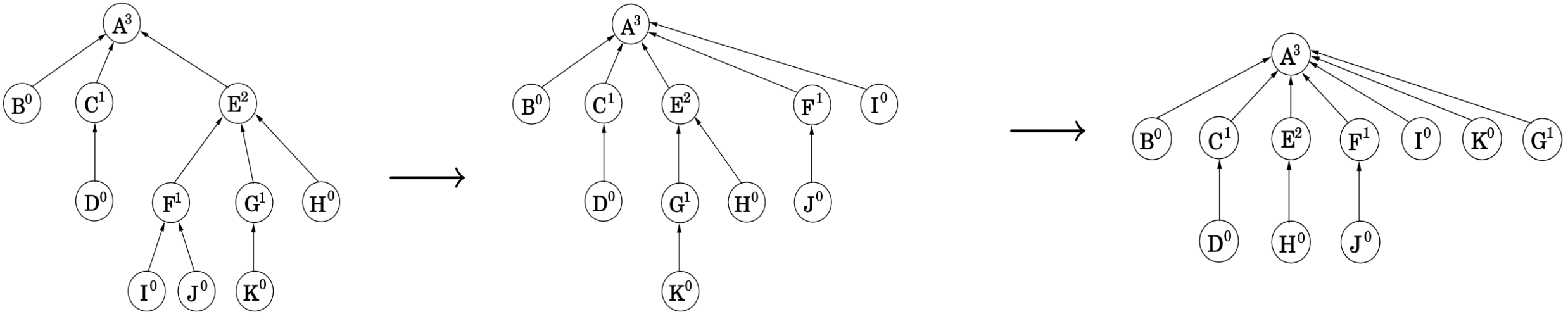
In Kruskal we do first two finds to find r_x, r_y; if $r_x \neq r_y$ then we do union(r_x, r_y) using the root nodes we already found, so we do each find once.

Path compression

union:
$2\times find + O(1)$

- Change all pointers during each find() so that they point directly to the root.

The effect of path compression: `find`(I) followed by `find`(K)



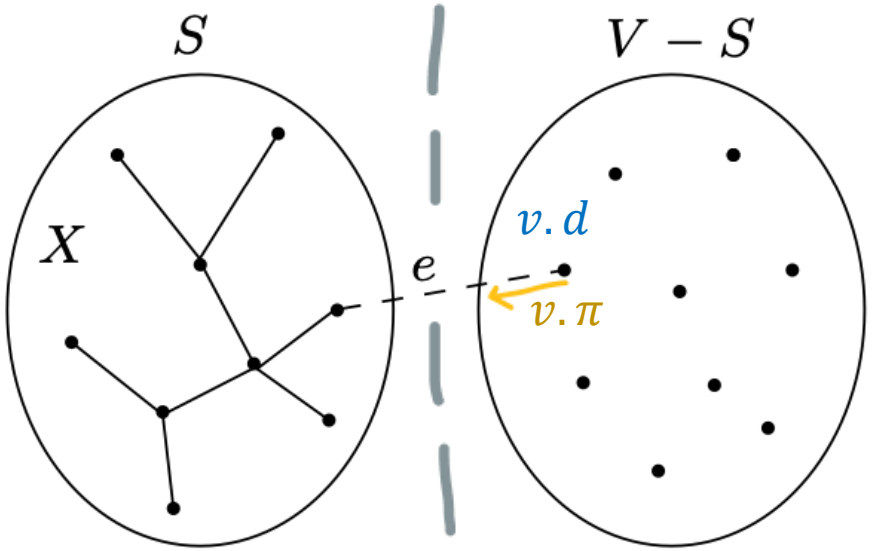
Amortized cost of union-find: $O(\log^* n) \approx O(1)$,
total time for $2m$ finds: $O(m \log^* n)$.

$\log^* 1000 = 4$:
$\log\log\log\log 1000 = 1$
$\log^* n > 5$ if $n \geq 2^{265536}$.

Prim's algorithm

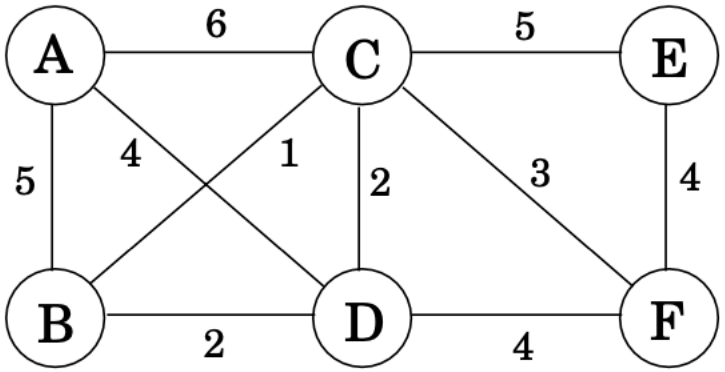
- **Prim's algorithm**: the intermediate set of edges X always forms a subtree, and S is chosen to be the set of this tree's vertices.
- Strongly reminiscent of Dijkstra's algorithm:
  - in Prim's, the value of a node is the weight of the lightest incoming edge from set S.
  - in Dijkstra's it is the length of an entire path to that node from the starting point.

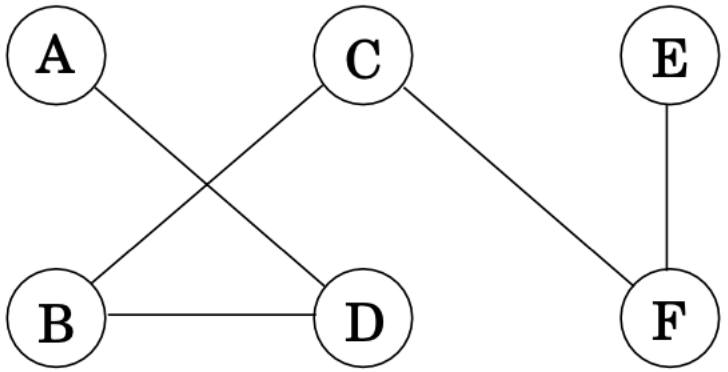
apply the cutting lemma



$v.d$ = smallest distance to any node in S.
$v.\pi$ = node in S that achieves this smallest distance.

Example





Set S	A	B	C	D	E	F
$\{\}$	$0/\text{nil}$	∞/nil	∞/nil	∞/nil	∞/nil	∞/nil
A		$5/A$	$6/A$	$4/A$	∞/nil	∞/nil
A, D		$2/D$	$2/D$		∞/nil	$4/D$
A, D, B			$1/B$		∞/nil	$4/D$
A, D, B, C					$5/C$	$3/C$
A, D, B, C, F					$4/F$	